

Universidad Nacional del Noroeste de la Provincia de Buenos Aires

UNNOBA

Ingeniería en Informática

Programación Lógica y Funcional

TRABAJO FINAL

**Sistema de apoyo a la decisión de fichajes de fútbol en Prolog,
con una implementación comparativa en Racket**

Estudiantes

Felipe Jorge Lucero

Renzo Ariel Pellizzari

Docente

Sabrina Pompeii

Junín, provincia de Buenos Aires — 2026

1. Título

Sistema de apoyo a la decisión de fichajes de fútbol en Prolog, con una implementación comparativa en Racket.

2. Resumen

El trabajo consistió en construir un sistema en Prolog que ayude a decidir qué jugadores puede comprar un club de fútbol. Representamos jugadores y equipos como hechos (posición, edad, precio, nacionalidad, rendimiento, presupuesto) y escribimos reglas que verifican si una incorporación es viable según el dinero disponible, el cupo de extranjeros, la edad y los puestos que el plantel necesita cubrir. Sobre esa base agregamos predicados que recomiendan candidatos, eligen el mejor fichaje individual y buscan la combinación de compras con mayor rendimiento conjunto, incorporando una medida de química entre jugadores para que el resultado no sea una simple suma de valores individuales. Para poder mostrar el sistema en funcionamiento armamos una interfaz web conectada al motor de inferencia. Por último, reescribimos las mismas consultas en Racket con estructuras inmutables y funciones de orden superior, lo que nos permitió comparar los dos paradigmas de la materia sobre un problema idéntico y no sobre ejemplos de manual. La conclusión principal es que Prolog resulta mucho más conciso para expresar restricciones, mientras que Racket obliga a programar la búsqueda y, a cambio, da más control sobre el costo del cómputo.

Palabras clave: programación lógica, Prolog, programación funcional, Racket, restricciones, sistemas de apoyo a la decisión.

3. Introducción

La idea del trabajo salió de una pregunta bastante concreta: cuando un club decide comprar un jugador, ¿qué está evaluando en realidad? La respuesta casi nunca es únicamente el rendimiento individual. Un director deportivo trabaja con un presupuesto cerrado, con un plantel que ya tiene huecos en determinados puestos, con reglamentos de liga que limitan cuántos extranjeros pueden inscribirse [9] y con la sospecha, difícil de cuantificar, de que hay jugadores que rinden mejor juntos que por separado.

Ese conjunto de condiciones nos pareció un buen caso para la programación lógica. En un lenguaje imperativo habría que escribir el recorrido de la lista de jugadores, el chequeo de cada restricción y la comparación entre alternativas. En Prolog alcanza con declarar qué condiciones vuelven válido un fichaje y dejar que el motor de inferencia se ocupe de encontrar las soluciones que las satisfacen.

El objetivo, entonces, fue modelar esas decisiones en Prolog para responder tres preguntas: qué jugadores puede fichar un equipo, cuál de todos conviene más y qué conjunto de incorporaciones maximiza el rendimiento sin romper ninguna restricción. Aclaramos desde el principio que la herramienta no reemplaza el criterio de un entrenador ni de un scout: filtra, ordena y justifica, pero la decisión sigue siendo de una persona.

De ese objetivo general se desprenden los objetivos específicos que ordenaron el desarrollo:

- Representar el dominio (jugadores, equipos, presupuestos y necesidades de plantel) como una base de hechos consultable.
- Expresar las restricciones del mercado de pases como reglas, de manera que agregar o modificar una condición no obligue a reescribir el resto del modelo.
- Resolver, sobre esa base, las consultas de viabilidad, recomendación, mejor fichaje individual y combinación óptima de incorporaciones.
- Construir una interfaz que permita ejecutar esas consultas sin conocer Prolog.
- Reimplementar el mismo conjunto de consultas en Racket y comparar ambos paradigmas sobre resultados concretos.

Al alcance inicial le agregamos dos cosas durante el desarrollo. La primera fue una interfaz web, porque al probar el sistema nos dimos cuenta de que leer las respuestas en la consola de SWI-Prolog vuelve casi imposible mostrarle el resultado a alguien que no conoce el lenguaje. La segunda fue reimplementar las consultas centrales en Racket, lo que nos dejó comparar el paradigma lógico con el funcional sobre el mismo dominio y con los mismos datos.

4. Marco teórico

4.1 El paradigma lógico

La programación lógica es un paradigma declarativo: el programa describe relaciones entre entidades y las condiciones bajo las cuales esas relaciones se cumplen, en lugar de indicar el procedimiento paso a paso que debe seguirse para calcularlas. Prolog implementa esta idea sobre un subconjunto del cálculo de predicados de primer orden, las cláusulas de Horn, y resuelve las consultas mediante resolución SLD [2], [3].

Una base de conocimiento en Prolog está formada por hechos y reglas. Un hecho afirma que una relación se cumple para determinados términos; una regla establece que una relación se cumple si se cumplen otras. Cuando se lanza una consulta, el motor intenta demostrar que el objetivo se deduce de las cláusulas disponibles. Dos mecanismos sostienen ese proceso: la unificación, que liga variables con términos compatibles, y el

backtracking, que deshace decisiones y prueba alternativas cuando una rama de la búsqueda falla [1], [2].

La consecuencia práctica es que el programador escribe el qué y el intérprete resuelve el cómo. Esa promesa, sin embargo, tiene letra chica que se nota apenas se sale de los ejemplos de manual. El orden en que se escriben las cláusulas y los objetivos dentro de una regla afecta el rendimiento y, en presencia de recursión, puede provocar que una consulta no termine aunque la formulación lógica sea correcta. El operador de corte, que descarta puntos de backtracking pendientes, permite controlar ese comportamiento a cambio de resignar parte de la pureza declarativa del programa [2], [3]. Prolog trabaja además bajo la hipótesis de mundo cerrado: lo que no puede demostrarse a partir de la base se considera falso, algo que en este trabajo importa porque un jugador ausente de los hechos es indistinguible de un jugador que no existe.

4.2 Restricciones y búsqueda combinatoria

Desde el punto de vista del modelado, una restricción es simplemente un predicado más que acota el conjunto de soluciones aceptables. Problemas como el de este trabajo (elegir un subconjunto de jugadores que respete un tope de presupuesto, un cupo de extranjeros y una lista de puestos a cubrir) se corresponden con la familia de problemas de satisfacción de restricciones y optimización combinatoria descrita en la literatura clásica de inteligencia artificial [10]. Conviene tener presente desde el diseño que el espacio de búsqueda de estos problemas crece de forma exponencial con la cantidad de candidatos, porque esa característica termina condicionando decisiones de implementación.

La recursión sobre listas es la herramienta habitual para los cálculos agregados que exige este tipo de modelo: costo total de una operación, cantidad de extranjeros en una alineación, suma de rendimientos [3].

4.3 El paradigma funcional

El enfoque funcional comparte con el lógico el carácter declarativo, pero organiza el cómputo de otra manera. En Racket, un dialecto de Scheme, el dominio se representa con datos inmutables y las transformaciones se expresan componiendo funciones puras mediante operaciones de orden superior como map, filter y foldl [8]. La diferencia central respecto de Prolog es que la estrategia de búsqueda deja de ser responsabilidad del intérprete y pasa a ser parte del programa: si hace falta recorrer todos los subconjuntos posibles, hay que generarlos explícitamente.

4.4 La capa de interacción

La interfaz gráfica no forma parte de los contenidos de la materia, pero sí condiciona la utilidad del sistema. Su función es traducir acciones del usuario en consultas y devolver

las respuestas en un formato legible, sin absorber lógica de negocio que debería vivir en la base de conocimiento. Para eso utilizamos Streamlit, una biblioteca de Python que permite construir aplicaciones web a partir de un script [7].

5. Desarrollo

5.1 Arquitectura del sistema

El sistema quedó dividido en tres capas. La primera es la base de conocimiento del dominio: hechos sobre jugadores (posición, edad, precio, nacionalidad, rendimiento) y sobre equipos (presupuesto disponible, plantel actual, puestos a reforzar). La segunda es la capa de reglas, donde vive toda la lógica de decisión. La tercera es la interfaz web, que se comunica con el intérprete a través de un puente entre Python y SWI-Prolog.

La separación no es un adorno arquitectónico. Nos permitió modificar reglas y volver a probarlas desde la consola sin tocar una línea de la aplicación, y evitó que criterios de negocio terminaran escondidos dentro del código de la interfaz, que fue una tentación permanente cada vez que había que resolver algo con apuro.

5.2 Modelado y predicados principales

Los hechos se cargan con predicados de aridad fija, uno por entidad, y sobre ellos construimos predicados auxiliares de acceso (`precio_jugador/2`, `edad_jugador/2`, `posicion_jugador/2`, `es_extranjero/2`). Esa capa intermedia parecía redundante al principio, pero resultó útil cuando cambiamos el formato de los hechos: sólo hubo que reescribir los accesos y no las reglas que dependían de ellos.

Sobre esa base se definen las cuatro consultas centrales del trabajo, que se detallan más adelante en la Tabla 1. La regla de viabilidad es la que sostiene a todas las demás y tiene la siguiente forma:

```
puede_firmar(Equipo, Jugador) :-  
    jugador(Jugador, Posicion, Edad, Precio, Nacionalidad, _),  
    equipo(Equipo, Presupuesto, _, Necesidades),  
    Precio <= Presupuesto,  
    member(Posicion, Necesidades),  
    edad_admisible(Equipo, Edad),  
    cupo_extranjeros_disponible(Equipo, Nacionalidad).
```

Listado 1. Predicado de viabilidad de un fichaje.

Vale la pena señalar lo que esta regla no dice: no indica en qué orden recorrer los jugadores, no lleva un contador, no arma ninguna lista de resultados. Toda esa maquinaria, que en un lenguaje imperativo habría que escribir, queda a cargo del motor de inferencia. Consultar los candidatos de un equipo es preguntar por las soluciones de la misma regla con la segunda variable libre.

El cuadro siguiente resume las consultas que el sistema resuelve y las restricciones que interviene cada una:

Predicado	Qué responde	Restricciones que interviene
puede_firmar/2	Si un equipo puede incorporar a un jugador determinado.	Presupuesto, necesidad de la posición, rango de edad, cupo de extranjeros.
recomendar/2	Qué jugadores son viables para un equipo, ordenados por rendimiento.	Las de puede_firmar/2, más el filtro por contexto (exclusión de clubes rivales).
mejor_fichaje/2	Cuál es el candidato individual de mayor rendimiento.	Las de recomendar/2.
combinacion_optima /2	Qué conjunto de incorporaciones maximiza el rendimiento conjunto.	Presupuesto total del conjunto, cupo de extranjeros del plantel resultante, tamaño máximo de la combinación, química entre pares.

Tabla 1. Consultas principales de la base de conocimiento.

La química entre jugadores fue el agregado más interesante del modelo. Definimos un predicado que asigna un valor de afinidad a determinados pares (por nacionalidad compartida, por complementariedad de posiciones o por antecedentes de haber jugado juntos) y un rendimiento ajustado que suma esa sinergia al total del conjunto. El efecto es que la combinación óptima deja de ser la suma de los mejores jugadores disponibles: en varias pruebas el sistema prefirió dos jugadores de rendimiento medio con buena afinidad antes que una figura suelta más cara.

5.3 Explosión combinatoria y control del costo

El primer problema serio apareció al probar combinacion_optima/2 con la base completa. Generar todos los subconjuntos posibles de una lista de candidatos es un cómputo exponencial, y con más de veinte jugadores la consulta dejaba de responder en un tiempo razonable para una demostración en clase.

Consideramos dos caminos. Uno era implementar una poda real, descartando ramas apenas el costo parcial superara el presupuesto. El otro era acotar el problema antes de generar. Terminamos aplicando principalmente el segundo, con una versión de la consulta pensada para la interfaz que toma únicamente los mejores candidatos por rendimiento y limita el tamaño máximo de la combinación a tres jugadores, que además es una cantidad realista para un mercado de pases. La decisión es explícitamente un compromiso: la respuesta que devuelve la versión acotada es óptima dentro del subconjunto considerado,

no dentro del universo completo de jugadores. Nos pareció preferible una respuesta correcta y acotada, con el recorte documentado, antes que una consulta que no termina.

También agregamos predicados de filtrado por contexto, como la exclusión de jugadores de clubes rivales directos, para que la interfaz y la lógica mostraran siempre el mismo conjunto de candidatos.

5.4 Interfaz e integración de datos externos

La aplicación web organiza el sistema en paneles: evaluación de un fichaje puntual, recomendaciones para un equipo, comparación entre candidatos y visualización del plantel. Los datos de plantillas y el material gráfico se tomaron de fuentes públicas [5], [6].

Acá nos encontramos con el error más difícil de rastrear de todo el trabajo. La interfaz mostraba, en algunos casos, la foto de un jugador junto al nombre de otro. El origen resultó ser una combinación de tres causas: filas mal parseadas cuando el nombre contenía comas, un emparejamiento que se hacía por nombre en lugar de por identificador, y una caché de imágenes que sobrevivía a los cambios en los datos. La solución consistió en parsear cada fila de forma más estricta, indexar las imágenes por identificador único, limpiar la caché al recargar los datos y devolver una imagen genérica cuando la asociación no puede verificarse. Descargamos además una copia local de fotos y escudos, porque no queríamos que la exposición dependiera de la conexión a internet del aula.

5.5 Pruebas y resultados

Las pruebas de la parte lógica fueron manuales: armamos un conjunto de consultas con resultado conocido de antemano y verificamos que el sistema fallara donde tenía que fallar, que suele ser más informativo que verificar que acierta donde debe acertar. Ese enfoque, aunque simple, nos sirvió para detectar dos reglas que devolvían soluciones de más porque faltaba una condición de posición.

Caso	Resultado esperado	Resultado obtenido
Jugador dentro de presupuesto y en un puesto necesitado	Éxito	Éxito
Precio superior al presupuesto disponible	Falla	Falla
Posición que el equipo no tiene entre sus necesidades	Falla	Falla (tras corregir la regla)
Extranjero con el cupo del plantel ya completo	Falla	Falla

Caso	Resultado esperado	Resultado obtenido
Equipo inexistente en la base de hechos	Falla	Falla
Combinación cuyo costo total excede el presupuesto	Falla	Falla

Tabla 2. Casos de prueba de la capa lógica.

El sistema responde correctamente a las cuatro consultas centrales y los tiempos de respuesta en la interfaz se mantienen dentro de lo aceptable con la versión acotada de la combinación óptima. Lo que no podemos afirmar es que las recomendaciones tengan valor futbolístico real: los rendimientos que cargamos provienen de una fuente comercial [6] y funcionan como dato de prueba, no como una medición seria del nivel de un jugador.

5.6 Implementación funcional en Racket

Para cubrir el componente funcional de la materia reimplementamos en Racket las mismas cuatro consultas. El dominio se representa con struct inmutables y las condiciones de fichaje se expresan como predicados puros que se combinan con filter; el cálculo de costos y rendimientos totales se resuelve con foldl.

La diferencia con Prolog se hizo evidente en la combinación óptima. Lo que allá era una consulta que delegaba la búsqueda en el motor, acá hubo que escribirlo: una función recursiva que genera los subconjuntos, un filtro de validez y una selección por máximo rendimiento. El código funcional es considerablemente más largo, pero también deja a la vista dónde se gasta el tiempo de cómputo, algo que en Prolog quedaba oculto detrás del backtracking. El módulo incluye pruebas con rackunit y una demostración por consola.

5.7 Comparación entre paradigmas

Con las dos implementaciones terminadas la comparación dejó de ser teórica. En Prolog, la traducción de una regla de negocio a código es casi directa: la condición "un equipo puede fichar a un jugador si le alcanza el presupuesto, necesita ese puesto y no supera el cupo de extranjeros" se escribe prácticamente con esas mismas palabras. La contrapartida es que, cuando algo falla, entender por qué una consulta no unifica lleva más tiempo que depurar una función.

En Racket el costo inicial es mayor, porque hay que decidir y programar la estrategia de recorrido, pero el flujo de datos queda explícito y las funciones se prueban de a una. Para las consultas simples Prolog nos resultó claramente más productivo; para la combinación óptima, donde el problema real era controlar el costo del cómputo, tener la búsqueda a la vista fue una ventaja.

Aspecto	Prolog	Racket
Expresión de restricciones	Directa: cada condición es un objetivo dentro de la regla.	Requiere escribir predicados y componerlos con filter.
Estrategia de búsqueda	A cargo del motor de inferencia; implícita.	Programada explícitamente por el desarrollador.
Control del costo de cómputo	Indirecto: orden de cláusulas, cortes, acotamiento previo.	Directo: la generación de candidatos está a la vista.
Depuración	Costosa cuando una consulta falla sin motivo evidente.	Más simple: cada función se prueba de forma aislada.
Extensión del código	Menor para las consultas simples.	Mayor, sobre todo en la combinación óptima.

Tabla 3. Comparación de ambas implementaciones sobre el mismo problema.

La conclusión que sacamos es que la elección depende de dónde esté la dificultad del problema. Si está en expresar relaciones y restricciones, conviene el enfoque lógico. Si está en controlar cómo se recorre el espacio de soluciones, el enfoque funcional da más herramientas.

6. Conclusiones

El trabajo nos permitió aplicar los contenidos de la materia a un problema que no venía resuelto de antemano, y esa fue probablemente la parte más valiosa. La fortaleza principal del enfoque lógico apareció en el modelado: expresar restricciones interdependientes en Prolog resultó considerablemente más económico que hacerlo de forma imperativa, y el código quedó legible incluso para quien no participó de escribirlo.

En lo personal, lo que más nos costó fue el cambio de forma de pensar. Durante las primeras semanas escribíamos Prolog como si fuera un lenguaje imperativo, tratando de controlar el orden de ejecución, y recién cuando dejamos de hacerlo el modelo empezó a simplificarse. Definir qué condiciones vuelven válida una solución y delegar la búsqueda es una disciplina que cuesta adoptar cuando uno viene de otros lenguajes.

También quedan cosas por mejorar, y conviene decirlas. El recorte del espacio de búsqueda en la combinación óptima es una limitación real y no una optimización: el sistema no garantiza el óptimo global. La base de conocimiento es chica y los datos de rendimiento son aproximados. Las pruebas de la parte lógica son manuales, a diferencia de las del módulo en Racket.

Como continuación nos parecen razonables tres líneas. Reemplazar la generación exhaustiva por una búsqueda con poda por presupuesto, que permitiría levantar el recorte

sin perder tiempos de respuesta. Enriquecer los hechos con métricas más serias, como minutos jugados o historial de lesiones. Y agregar un criterio multiobjetivo, porque en la práctica un club rara vez optimiza rendimiento solamente: también le importan la edad del plantel y el valor de reventa.

7. Bibliografía

- [1] SWI-Prolog, “SWI-Prolog Reference Manual,” 2026. [En línea]. Disponible en: <https://www.swi-prolog.org/pldoc/>
- [2] I. Bratko, Prolog Programming for Artificial Intelligence, 4.a ed. Harlow, Reino Unido: Pearson, 2011.
- [3] W. F. Clocksin y C. S. Mellish, Programming in Prolog, 5.a ed. Berlín, Alemania: Springer, 2003.
- [4] P. Blackburn, J. Bos y K. Striegnitz, “Learn Prolog Now!,” [En línea]. Disponible en: <http://www.learnprolognow.org/>
- [5] Transfermarkt, “FC Barcelona – Club profile,” 2026. [En línea]. Disponible en: <https://www.transfermarkt.com/fc-barcelona/startseite/verein/131>
- [6] EA Sports, “EA Sports FC Player Ratings,” 2026. [En línea]. Disponible en: <https://www.ea.com/games/ea-sports-fc/ratings>
- [7] Streamlit, “Streamlit Documentation,” 2026. [En línea]. Disponible en: <https://docs.streamlit.io/>
- [8] Racket, “The Racket Guide,” 2026. [En línea]. Disponible en: <https://docs.racket-lang.org/guide/>
- [9] FIFA, “Regulations on the Status and Transfer of Players,” 2026. [En línea]. Disponible en: <https://www.fifa.com/legal>
- [10] S. Russell y P. Norvig, Artificial Intelligence: A Modern Approach, 4.a ed. Hoboken, NJ, EE. UU.: Pearson, 2021.